

CUDA syntax primer

Referência de sintaxe, do zero. Não é pra ler de uma vez — é pra consultar enquanto você mexe nos exercícios de [learning/phase-02-gpu-architecture/](#). Cada seção tem um trechinho pequeno e correto que você pode copiar direto.

1. Anatomia de um arquivo `.cu`

Um único arquivo `.cu` mistura **código de host (CPU)** e **código de device (GPU)**. O `nvcc` separa os dois automaticamente na compilação: o que tem qualificador `__global__`/`__device__` vira código de GPU: o resto é C++ normal de CPU.

```
#include <cstdio>
#include <cuda_runtime.h>

__global__ void meuKernel() {           // roda na GPU
    printf("oi da thread %d\n", threadIdx.x);
}

int main() {                            // roda na CPU (host)
    meuKernel<<<1, 4>>>();              // CPU manda a GPU executar
    cudaDeviceSynchronize();           // CPU espera a GPU terminar
    return 0;
}
```

Compila com `nvcc arquivo.cu -o programa`, roda com `./programa` igual qualquer binário C++.

2. Qualificadores de função

Qualificador	Roda em	Chamado de	Uso
<code>__global__</code>	GPU	Host (via <code><<<>>></code>)	O kernel em si — ponto de entrada
<code>__device__</code>	GPU	Outro código de GPU	Função auxiliar chamada de dentro de um kernel
<code>__host__</code>	CPU	Host	Igual função C++ normal (é o padrão se você não escrever nada)
<code>__host__</code> <code>__device__</code>	Ambos	Ambos	Compila duas versões — útil pra código matemático reusado dos dois lados

```
__device__ float quadrado(float x) { // helper, so pode ser chamado de outro
codigo de GPU
    return x * x;
}
```

```

}

__global__ void somaQuadrados(float* a, float* b, float* c) {
    int i = threadIdx.x;
    c[i] = quadrado(a[i]) + quadrado(b[i]); // chamando o __device__ daqui
}

```

Regra importante: `__global__` sempre retorna `void`. Se seu kernel precisa "retornar" algo, ele escreve o resultado num ponteiro de memória que você passou como argumento — não existe `return valor;` num kernel.

3. Sintaxe de lançamento

```
kernel<<<gridDim, blockDim>>>(args);
```

`gridDim` e `blockDim` podem ser um `int` simples (1D) ou um `dim3` (até 3D):

```

meuKernel<<<10, 256>>>(...);           // 1D: 10 blocks de 256 threads

dim3 block(16, 16);                     // 2D: 16x16 = 256 threads por block
dim3 grid(4, 4);                         // 2D: 4x4 = 16 blocks
meuKernel2D<<<grid, block>>>(...);

```

Forma completa (4 parâmetros, os 2 últimos opcionais):

```
kernel<<<grid, block, sharedMemBytes, stream>>>(args);
```

- `sharedMemBytes`: bytes extras de shared memory alocados dinamicamente (0 se você só usa `__shared__` estático — ver seção 6).
- `stream`: em qual fila de execução assíncrona rodar (fora de escopo por enquanto — default é 0, a stream padrão).

4. Variáveis embutidas

Disponíveis automaticamente **dentro** de qualquer `__global__`/`__device__`, sem você declarar nada. Todas são do tipo `dim3` (ou `uint3`), com componentes `.x`, `.y`, `.z`:

```

__global__ void mostraIndices() {
    printf("thread (%d,%d,%d) do block (%d,%d,%d) | blockDim=(%d,%d,%d) gridDim=(%d,%d,%d)\n",
           threadIdx.x, threadIdx.y, threadIdx.z,
           blockIdx.x, blockIdx.y, blockIdx.z,
           blockDim.x, blockDim.y, blockDim.z,

```

```
        gridDim.x, gridDim.y, gridDim.z);
    }
```

Variável	O que é
<code>threadIdx</code>	Posição da thread dentro do seu block (0-indexado)
<code>blockIdx</code>	Posição do block dentro do grid (0-indexado)
<code>blockDim</code>	Tamanho do block que você lançou (quantas threads por block)
<code>gridDim</code>	Tamanho do grid que você lançou (quantos blocks)

O índice global 1D (o mais comum que você vai escrever) é sempre esta conta:

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

5. API de memória

A GPU tem memória própria (VRAM), separada da RAM da CPU. Você precisa alocar, copiar pra lá, processar, copiar de volta, e liberar — manualmente.

```
int n = 1024;
size_t bytes = n * sizeof(float);

float* h_data = new float[n];           // h_ = host (RAM da CPU)
float* d_data;                           // d_ = device (VRAM da GPU)

cudaMalloc(&d_data, bytes);              // aloca na GPU
cudaMemcpy(d_data, h_data, bytes, cudaMemcpyHostToDevice); // CPU -> GPU

meuKernel<<<blocks, threads>>>(d_data);

cudaMemcpy(h_data, d_data, bytes, cudaMemcpyDeviceToHost); // GPU -> CPU

cudaFree(d_data);                        // libera a VRAM
delete[] h_data;
```

Função	Faz
<code>cudaMalloc(&ptr, bytes)</code>	Aloca <code>bytes</code> na VRAM, guarda o endereço em <code>ptr</code>
<code>cudaMemcpy(dst, src, bytes, direção)</code>	Copia memória; direção é um dos 4 abaixo
<code>cudaMemset(ptr, valor, bytes)</code>	Zera (ou preenche) memória da GPU, tipo <code>memset</code>
<code>cudaFree(ptr)</code>	Libera memória da VRAM

Direções do `cudaMemcpy`: `cudaMemcpyHostToDevice`, `cudaMemcpyDeviceToHost`, `cudaMemcpyDeviceToDevice`, `cudaMemcpyHostToHost`.

Existe também `cudaMallocManaged` (unified memory) — aloca um ponteiro que CPU e GPU podem acessar diretamente, sem `cudaMemcpy` manual (o driver migra os dados por baixo dos panos). Mais simples de escrever, mas esconde exatamente o custo de transferência que você quer aprender a enxergar agora — por isso os exercícios usam `cudaMalloc`+`cudaMemcpy` explícito.

6. Qualificadores de memória

```
__global__ void exemplo(float* global_data) {
    __shared__ float cache[256];          // shared memory: por-block, rapida, voce
    gerencia

    extern __shared__ float dinamica[]; // shared memory de tamanho dinamico
                                     // (extern e obrigatorio aqui; o tamanho
                                     // em bytes vem do 3o parametro do
    <<<>>>)

    float local = global_data[threadIdx.x]; // sem qualificador = registrador (ou
    local se nao couber)
}

__constant__ float coeficientes[16]; // memoria constante: read-only, cacheada,
visivel a todos os kernels
```

Qualificador	Onde vive	Escopo	Quando usar
(nenhum, variável local)	Registrador (ou memória local se não couber)	1 thread	Padrão — a maioria das suas variáveis
<code>__shared__</code>	Shared memory / L1 da SM	Todas as threads do mesmo block	Cooperação entre threads (ver capítulo 07 da teoria)
<code>__constant__</code>	Memória constante (cacheada)	Todos os kernels, read-only do device	Coeficientes/parâmetros pequenos e fixos
(ponteiro passado como argumento)	Memória global (VRAM)	Toda a GPU, todos os kernels	Seus dados de entrada/saída

`__restrict__` não é memória, é uma dica de otimização num parâmetro ponteiro: promete ao compilador que dois ponteiros nunca apontam pra região sobreposta, permitindo mais otimização (`float* __restrict__ out`).

7. Sincronização

```
__global__ void exemplo() {
    __shared__ int cache[256];
    int tid = threadIdx.x;

    cache[tid] = tid;
    __syncthreads();      // TODAS as threads do block esperam aqui antes de
    continuar

    int vizinho = cache[(tid + 1) % 256]; // agora seguro: todo mundo ja escreveu
}
```

Primitiva	Espera	Escopo
<code>__syncthreads()</code>	Todas as threads do block chegarem nesse ponto	Block inteiro (todos os warps dele)
<code>__syncwarp()</code>	Todas as lanes ativas do warp reconvergiem	1 warp
<code>atomicAdd(ptr, val)</code> (e variantes: <code>atomicSub</code> , <code>atomicMax</code> , <code>atomicCAS...</code>)	N/A — não é barreira, é uma operação indivisível	Qualquer escopo de memória (global ou shared)

Cuidado com `__syncthreads()` dentro de `if`: se só parte das threads do block entra no `if`, e a barreira está lá dentro, o programa trava (deadlock) — todas as threads do block precisam alcançar a mesma `__syncthreads()`, sem exceção.

8. Checagem de erro

Chamadas CUDA **não lançam exceção** quando falham — elas retornam um `cudaError_t` que você precisa checar manualmente. Sem isso, um kernel que falha silenciosamente parece ter rodado normal.

```
#define CUDA_CHECK(call) \
    do { \
        cudaError_t err = call; \
        if (err != cudaSuccess) { \
            fprintf(stderr, "CUDA erro em %s:%d: %s\n", \
                __FILE__, __LINE__, cudaGetErrorString(err)); \
            exit(1); \
        } \
    } while (0)

int main() {
    float* d_data;
    CUDA_CHECK(cudaMalloc(&d_data, 1024 * sizeof(float)));

    meuKernel<<<10, 256>>>(d_data);
    CUDA_CHECK(cudaGetLastError()); // pega erro de LANCAMENTO do kernel
    CUDA_CHECK(cudaDeviceSynchronize()); // pega erro de EXECUCAO do kernel
}
```

```

    CUDA_CHECK(cudaFree(d_data));
    return 0;
}

```

Repare: erro de lançamento (`cudaGetLastError`, ex: configuração de grid inválida) é diferente de erro de execução (`cudaDeviceSynchronize`, ex: acesso de memória fora dos limites dentro do kernel) — os dois precisam ser checados separadamente.

Os exercícios que fizemos até agora **não têm** essa checagem, de propósito, pra manter o código enxuto enquanto o foco era conceito. Vale adicionar quando você for escrever algo mais sério (é exatamente o que o `KERNEL_WORKFLOW.md` cobra na fase de correctness).

9. Padrões comuns

Boundary check — quando o número de threads lançadas não bate exato com o tamanho do problema (quase sempre):

```

__global__ void kernel(float* data, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) { // threads "sobrando" do ultimo block nao fazem nada
        data[i] = data[i] * 2.0f;
    }
}

```

Grid-stride loop — quando o problema é maior que o grid que você lançou (cada thread processa vários elementos, pulando de `gridDim.x * blockDim.x` em `gridDim.x * blockDim.x`):

```

__global__ void kernel(float* data, int n) {
    int stride = gridDim.x * blockDim.x;
    for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < n; i += stride) {
        data[i] = data[i] * 2.0f;
    }
}

```

Isso desacopla "quantas threads eu lancei" de "quantos elementos existem" — útil quando `n` só é conhecido em runtime e pode ser gigante.

Onde ver isso funcionando de verdade

- [cuda-lab/fundamentals/01-vector-add/vector_add.cu](#) — seções 1-4 juntas
- [learning/phase-02-gpu-architecture/exercises/code/](#) — os 15 exercícios usam praticamente toda seção deste guia, cada um isolando um pedaço